

Requirement 4

Design 1

Create class/interface:

1. CoatWeaponAction
2. Poisoning
3. Frosting
4. Snowcoating
5. Yewberrycoating
6. Coatable interface
7. Coating interface
8. CountinuousWarmthloss

Pro	Cons
Implementing the same interface, each coating only needs to define its own behaviour, like how it poisons or freezes the target.	If both YewberryCoating and SnowCoating implement the same Coating interface but still repeat similar internal logic, such as stacking, duration handling, and round decrementing, this design breaks the DRY principle and reduces code maintainability.
	Since ContinuousDamage already handles turn-based effects, creating another ContinuousWarmthLoss is unnecessary and breaks the DRY and Single Responsibility principles.
	Although new coatings can still implement the same Coating interface, the current design lacks a shared abstract layer for common logic like duration and stacking.

Design 2

Create class

1. CoatWeaponAction
2. Poisoning
3. Frostbiting
4. CountinuousWarmthloss
5. CountinuousEffect
6. Snowcoating
7. YewBerrycoating
8. Coatable interface

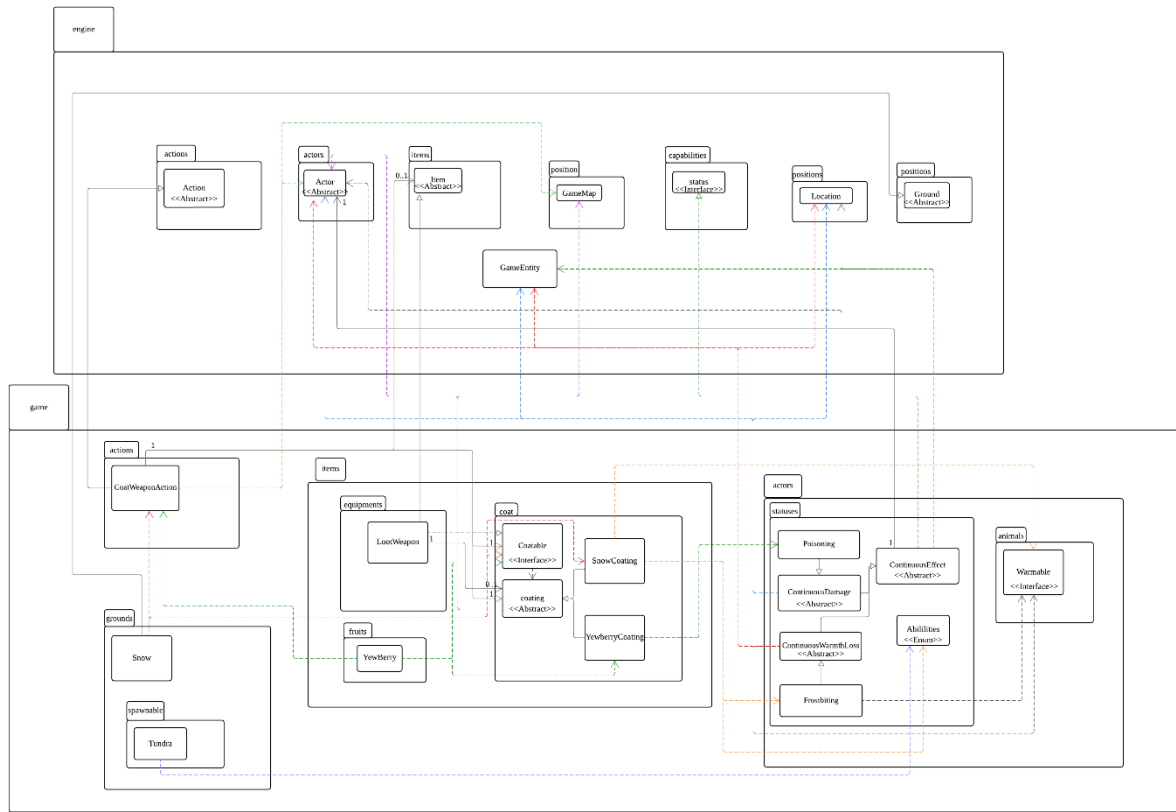
9. Coating abstract

Pro	Cons
Using an abstract Coating class allows both YewberryCoating and SnowCoating to share common logic such as duration, stacking, and expiry, which followed the DRY and Open/Closed principles because new coatings can be added by extending the abstract class without changing existing code.	For the current scope with only two coatings, the design may seem slightly over-engineered. The abstract structure adds extra complexity at this stage, but its advantages—such as easier extension and better code reuse. it will become more valuable when more coating types or coatable targets are introduced in the future.
The design achieves the DRY principle by using a shared abstract class called ContinuousEffect. This class centralises common logic such as ticking, duration handling, stacking, and expiry. Subclasses like ContinuousDamage and ContinuousWarmthLoss only need to define how the specific effect is applied, which helps reuse code and keeps the behaviour consistent across different continuous effects.	
The design keeps the coupling low by following the Dependency Inversion and Single Responsibility principles. CoatWeaponAction depends on the abstract types Coating and Coatable instead of concrete classes. This means weapons only handle combat, coatings handle their own effects, and the action simply coordinates them. Each class therefore has a single, clear responsibility, making the system easier to maintain and extend.	

Choose:

Design 2 was chosen because it better follows the SOLID principles, providing a cleaner and more maintainable structure compared to Design 1. By introducing an abstract Coating class and a Coatable interface, the design separates responsibilities clearly and removes code duplication. It follows the Single Responsibility Principle by ensuring each class focuses on one task, like coatings handle effects, actions manage coating application, and weapons handle combat. It also supports the Open/Closed Principle, as new coatings can be added by extending the abstract class without modifying existing code. Furthermore, the design respects the Dependency Inversion Principle, since CoatWeaponAction depends on abstractions rather than concrete classes. Although this approach is slightly more complex, it achieves better

flexibility, scalability, and long-term maintainability, making Design 2 the more appropriate and sustainable choice for REQ4.



1. What classes will exist in the extended system
 - a. Interface:
 - i. Coatable
 - ii. Warmable
 - b. Abstract class:
 - i. ContinuousEffect
 - ii. ContinuousWarmthloss
 - iii. continuousDamage
 - iv. Coating
 - c. Enum class:
 - i. Abilities
 - d. Concrete Class:
 - i. CoatWeaponAction
 - ii. SnowCoating
 - iii. YewberryCoating
 - iv. LootWeapon
 - v. YewBerry
 - vi. Frosting
 - vii. Poisoning

viii. Snow

ix. Tundra

2. Roles & Responsibilities

- a. Coatable → Represents any object that can receive a coating, usually a weapon, and defines how it interacts with different coating effects in the game.
- b. ContinuousEffect → Acts as an abstract base class that controls continuous effects in the game, handling shared logic like duration, stacking, and expiry for different status types.
- c. ContinuousWarmthLoss → A subclass of ContinuousEffect that reduces an actor's warmth each turn, showing the effect of cold conditions or frostbite in the game.
- d. ContinuousDamage → A subclass of ContinuousEffect that deals damage to the actor every turn, representing continuous effects like poison or bleeding.
- e. Abilities → Stores unique capability flags for actors (e.g., COLD_RESISTANT). Used to determine immunity or resistance when applying coating effects like Frosting.
- f. CoatWeaponAction → Handles the action of coating a weapon, letting the actor apply a chosen coating to a coatable item and updating the weapon's status accordingly.
- g. SnowCoating → Extends the Coating class and applies a frostbite effect to targets, causing continuous warmth loss for a few turns after being attacked.
- h. YewberryCoating → Extends the Coating class and applies a poisoning effect to the target, dealing continuous damage for several turns after being hit.
- i. LootWeapon → Represents a weapon that can be coated with different effects, working together with the Coating and Coatable system to allow players to enhance attacks with special status effects.
- j. YewBerry → Acts as a consumable item that provides the YewBerryCoating when used, letting the player coat a weapon to apply poison effects on future attacks.
- k. Frostbiting → A continuous status effect that reduces an actor's warmth each turn, used by SnowCoating to simulate the cold damage caused by frostbite.
- l. Poisoning → A continuous status effect that deals damage over several turns, used by YewBerryCoating to represent the poison applied to targets after being hit.
- m. Snow → Represents a ground source that can be used to coat weapons with SnowCoating, allowing the player to apply frostbite effects on the weapons.
- n. Tundra → Acts as a special ground type that spawns cold-resistant creatures, giving them abilities that make them immune to frostbite effects from SnowCoating in the coating system.

3. How these classes relate to and interact with the existing system.

The coating system is closely connected to the existing combat, item, and status structures in the game. The Coatable interface works with the existing LootWeapon class so that weapons can be coated without changing their original attack logic. The CoatWeaponAction class extends the Action system and lets the actor apply a coating from either a consumable item, such as YewBerry, or from an environmental source like Snow. When the coating is applied, it interacts with the weapon through polymorphism, allowing any class that implements Coatable to receive a coating effect.

The abstract Coating class integrates with the continuous effect framework. It stores common logic such as duration, stacking, and expiry that connects directly to the existing status update cycle in the engine. Specific coatings, like YewBerryCoating and SnowCoating, extend this base class and interact with the status system through Poisoning and Frostbiting. These status effects are subclasses of ContinuousDamage and ContinuousWarmthLoss, which both inherit their turn-based ticking behaviour from ContinuousEffect.

Environmental classes also connect to this system. Tundra creates cold-resistant creatures with the COLD_RESISTANT ability, making them immune to frostbite from SnowCoating, while the Snow ground provides a natural coating source for the player.

4. How the classes interact to deliver functionality.

When the player performs the coating action during gameplay, the process begins with CoatWeaponAction. The actor selects a coating source, either from a YewBerry item or directly from the Snow ground. The action then checks whether the chosen weapon implements Coatable. If it does, the coating is applied, replacing any previous one. The abstract Coating class handles this replacement logic and keeps information such as effect type, duration, and remaining turns.

During combat, when a coated weapon is used to attack, the coating effect activates automatically. For instance, YewBerryCoating poisons the target, while SnowCoating applies frostbite. These effects create corresponding status objects, Poisoning and Frostbiting, which are attached to the target actor. Each turn, the engine's status update system triggers their tick() method through ContinuousEffect, applying poison damage or warmth reduction until the duration ends. The environment also plays a role in how this functionality appears in the game. Snow provides a renewable coating source, while creatures born in Tundra possess the COLD_RESISTANT ability, which prevents frostbite from being applied.

Overall, the functionality flows through clear stages: the player executes an action, the coating is applied through abstraction, and the resulting continuous effects are processed by the existing engine each turn. This chain of interaction demonstrates how the design not only extends existing mechanics but also keeps them consistent and modular.